

Object Oriented Analysis and Design

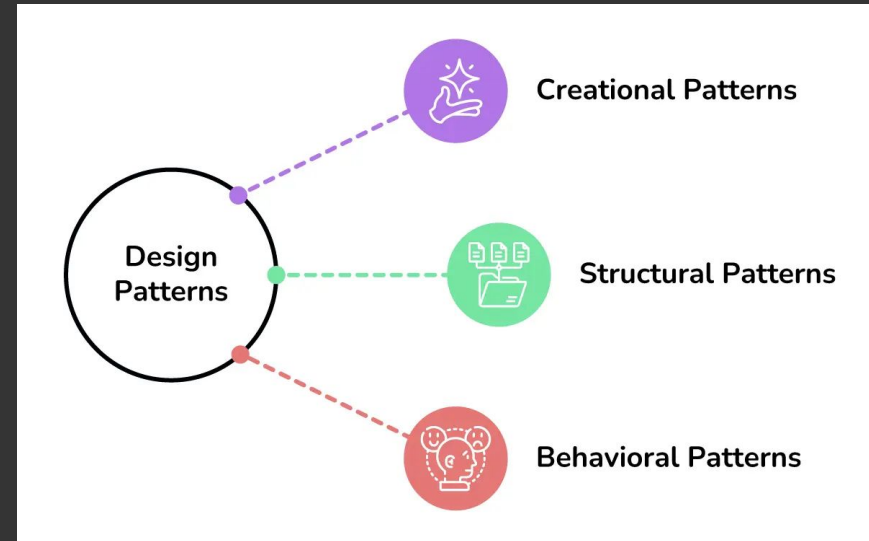
Design Patterns: Proxy and Observer

Vladimir Soroka



3 Categories Of Patterns

- **Creational** – object creation
- **Structural** – class/object composition
- **Behavioral** – object interaction & responsibility



What are we covering?

- **Proxy** — structural
- **Observer** - behavioral



The Proxy Pattern

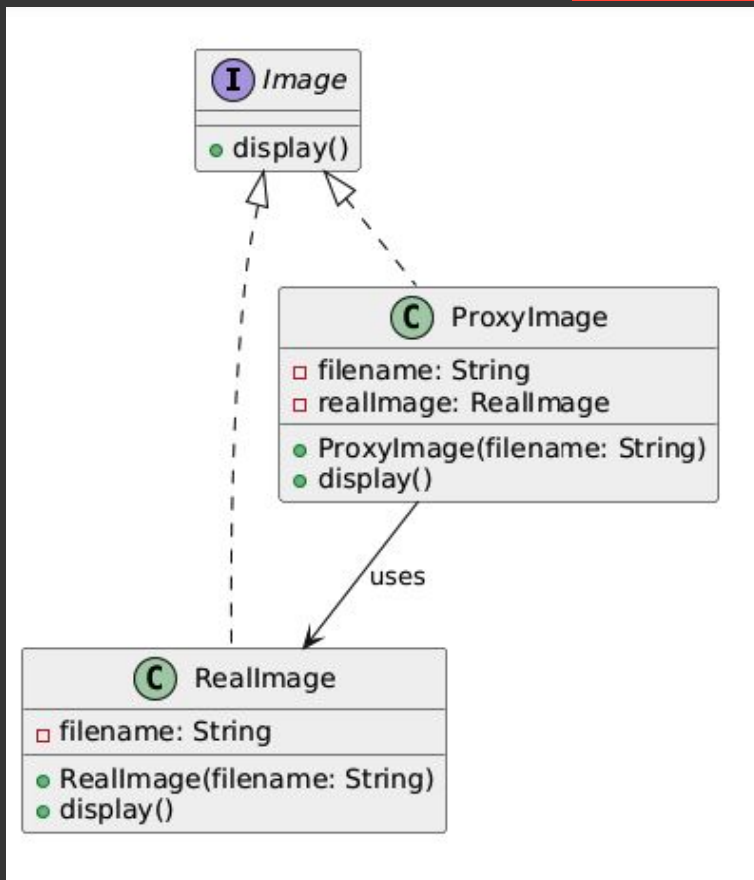
- **Purpose:**
 - Acts as a placeholder for another object
 - Controls access (load lazily, secure, remote access)
- **Example:** image viewer, database connections

Real World Analogy: "Think of a an executive assistant who filters visitors before letting them see the CEO."



Proxy UML

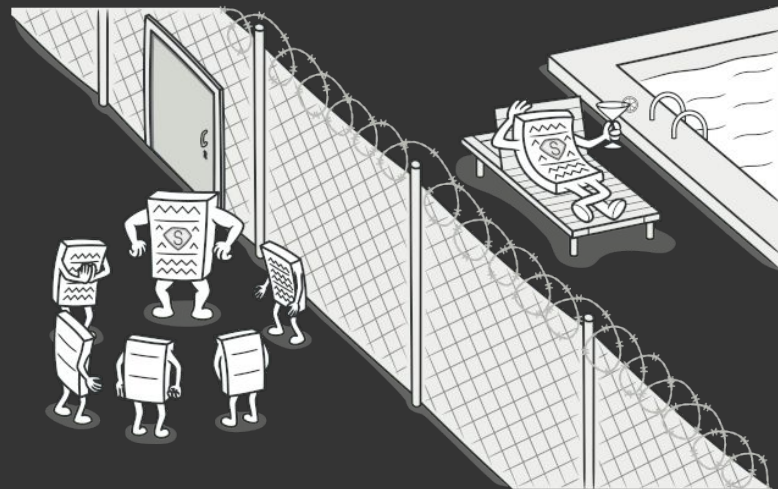
```
@startuml
interface Image {
    + display()
}
class RealImage implements Image {
    - filename: String
    + RealImage(filename: String)
    + display()
}
class ProxyImage implements Image {
    - filename: String
    - realImage: RealImage
    + ProxyImage(filename: String)
    + display()
}
ProxyImage --> RealImage : uses
@enduml
```



Proxy Code - Example

When to Use Proxy?

- **Virtual Proxy:** defer loading large objects
- **Protection Proxy:** restrict access
- **Remote Proxy:** access over a network
- **Smart Proxy:** reference counting/logging



What's Bad About Proxy?

1. Added Complexity

Proxy introduces an extra layer of indirection.

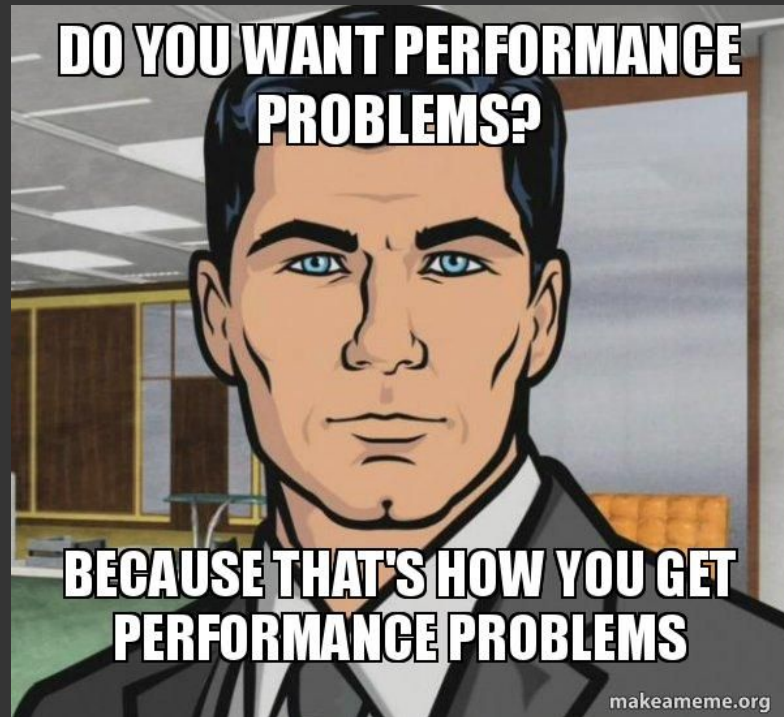
- Every call goes through a middleman, which makes the code harder to trace and debug.
- It requires managing two classes (Proxy and RealSubject) instead of one.
- *If your team inherits a system full of proxies, will it be easy to understand how the flow works?*

2. Performance Overhead

May degrade performance instead of optimizing it.

- The indirection adds function call overhead.
- Some proxies (like smart proxies with logging, counting, etc.) can become heavy.
Virtual Proxies that delay real object creation might involve costly checks every time.

⚠ *Irony:* The pattern meant to optimize performance (e.g., with lazy loading) can slow things down in the wrong hands.



What's Bad About Proxy?

3. Violation of the Single Responsibility Principle (SRP)

A proxy often does more than one thing.

- It's supposed to control access — but sometimes it logs, authenticates, caches, etc.
This bloats the Proxy class into a God Object.

💬 *Better Practice:* Delegate those responsibilities to separate decorators or middleware.

4. Tight Coupling to RealSubject

Proxies often depend on the concrete implementation.

- Especially in C++ or Java, Proxy usually directly creates the RealSubject (e.g., `new RealSubject()`).
- This makes testing, mocking, or swapping implementations harder.

💡 *Better:* Use dependency injection to decouple them.



What's Bad About Proxy?

5. Proxy Becomes a Trap for Overengineering

Sometimes devs use proxies when simpler patterns (like decorators or strategy) would suffice.

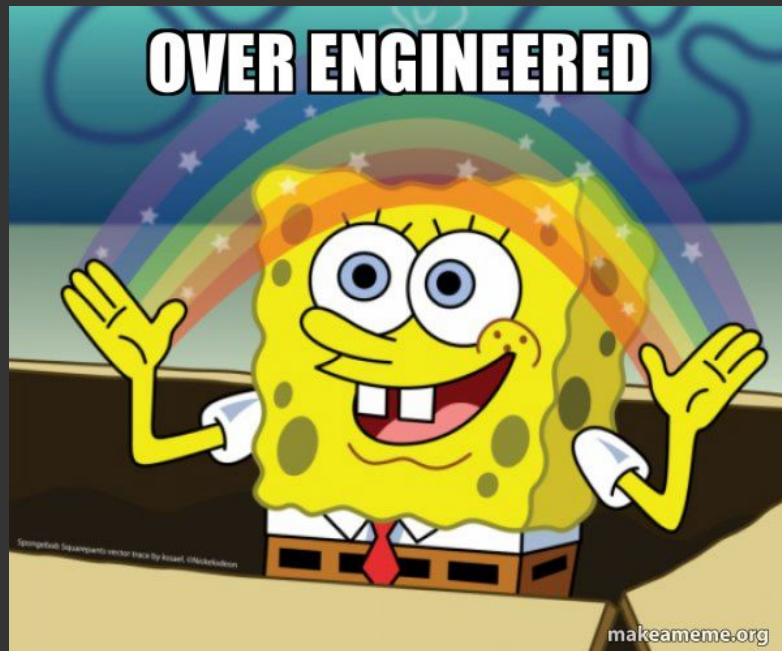
- Developers might prematurely optimize with a Proxy
- A Virtual Proxy might be overkill for small resources or lightweight services

6. Hides Underlying Failures

In remote proxies (e.g., network or DB), you may not realize you're working with a remote object.

- Looks like a local object, but introduces latency or failure risk. Exception handling becomes non-obvious.

 **Result:** Broken encapsulation of system behavior.



Proxy Summary



When to Avoid Proxy

- You don't need access control or lazy-loading.
- The real object is cheap and fast to create/use.
- The logic you're putting in the Proxy could be better expressed via composition or delegation.



When It's Justified

- You're deferring object creation (e.g., image loading).
- You need access control/security around an API.
- You're interacting with remote services.

The Observer Pattern

Purpose

- One-to-many dependency
- Subject notifies observers when state changes
- Example: UI button clicks, stock market trackers



Real World Analogy: “A radio station broadcasting to subscribers.”

Observer UML

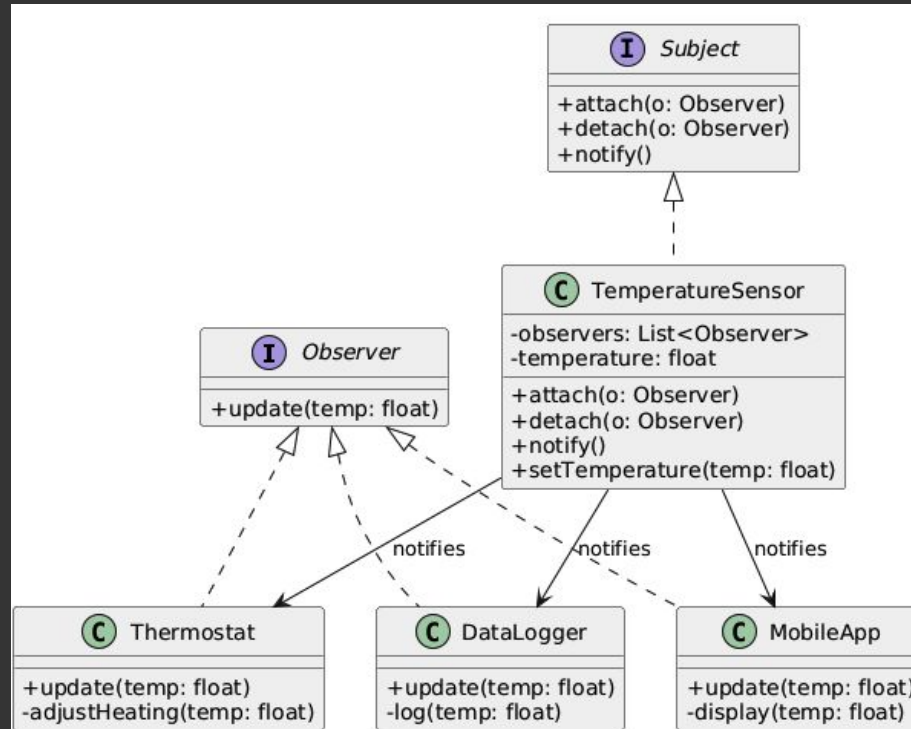
```
@startuml
    ' Subject implementation
    class TemperatureSensor {
        - observers: List<Observer>
        - temperature: float
        + attach(o: Observer)
        + detach(o: Observer)
        + notify()
        + setTemperature(temp: float)
    }
    Subject <|.. TemperatureSensor

    ' Interfaces
    interface Observer {
        + update(temp: float)
    }

    interface Subject {
        + attach(o: Observer)
        + detach(o: Observer)
        + notify()
    }

    ' Observers
    class Thermostat {
        + update(temp: float)
        - adjustHeating(temp: float)
    }
    class MobileApp {
        + update(temp: float)
        - display(temp: float)
    }
    class DataLogger {
        + update(temp: float)
        - log(temp: float)
    }
    Observer <|.. Thermostat
    Observer <|.. MobileApp
    Observer <|.. DataLogger
    TemperatureSensor --> Thermostat : notifies
    TemperatureSensor --> MobileApp : notifies
    TemperatureSensor --> DataLogger : notifies
@enduml
```

Observer UML



Observer Code - Example

What's Bad About Observer?

1. Hidden Dependencies (Tight Coupling Through Interfaces)

“Just because the Subject and Observer are loosely coupled *in code* doesn't mean they're loosely coupled *in behavior*.”

- Observers may depend on side effects triggered by the subject.
- You can't easily see who is subscribed — makes the flow of execution **non-obvious**.

💡 *Hard to trace the ripple effect of a `notify()` call.*

2. Unpredictable Execution Order

“Who gets notified first? What if it matters?”

- The order of observer notifications is typically undefined.
- If two observers depend on each other's state, it can lead to **race conditions** or **undefined behavior**.

🧠 *Lesson: avoid interdependent observers.*



Refactoring
code frequently
and staying
ahead of dependency
deprecations.



Get a new
job every 2
years to avoid
dealing with
my own tech debt.

What's Bad About Observer?

3. Memory Leaks from Forgotten Detach

“You added the observer... did you remember to remove it?”

- Especially in C++: dangling pointers if detach() isn't called.
- In garbage-collected languages: memory leaks if strong references are held.

💡 *This is one of the most common Observer-related bugs in C++.*

4. Performance Issues at Scale

“1000+ observers on one subject? Good luck debugging the slowdown.”

- Notifying a large number of observers can become expensive.
- No built-in prioritization or batching mechanism in classic implementations.

⚠️ *You may need a custom event bus or throttling.*



What's Bad About Observer?

5. Tight Observer Logic Embedded in UI or Domain Code

“Observers often do too much.”

- They handle data, update UI, call APIs, log actions — all in `update()`.
This violates the **Single Responsibility Principle**.



Code becomes brittle and hard to test.

6. Difficult to Debug and Test

“What triggered this observer? Why was `update()` called?”

- Hard to replicate and step through asynchronous notification chains.
Observers may fire off more observers — like a **domino effect**.



Unit testing is harder when you can't isolate the trigger flow.



Observer - Summary

When to Avoid Observer

- When updates must occur in a strict order
- When object lifetime is hard to control (e.g., multi-threaded systems)
- When memory management is manual (e.g., in C++ without smart pointers)
- When you can use **event queues, signals/slots, or publish-subscribe** instead

When It Still Works Well

- UI frameworks (e.g., Qt's signals and slots)
- Logging, notification, metrics
- Systems where **"fire and forget"** is acceptable

Bad Examples



Example 1

```
class OverloadedProxy : public Service {
    RealService* real;
public:
    void performAction() override {
        log();
        authenticate();
        cacheCheck();
        if (!real) real = new RealService();
        real->performAction();
        audit();
    }
};
```

Example 2

```
ProxyService::ProxyService() {  
    real = std::make_unique<RealService>();  
}
```

Example 3

```
void Proxy::calculateDiscount() {  
    if (userType == "VIP") return 0.2;  
    else return 0.1;  
}
```

Observer Examples

